敏捷開發佐證說明書

本次參與「2026 第 31 屆大專校院資訊應用服務創新競賽」之專題（專題名稱為：巧級——可驗證難度的空間邏輯訓練系統），使用看板方法並結合 Scrum 的迭代與回顧精神進行開發，符合「鈦坦敏捷開發特別獎」之作品要求，其佐證說明如下所示。

本專題作品採用「敏捷開發」之佐證說明

一、選擇參賽主題

本作品是一套以七巧板為載體的空間邏輯訓練系統：題目由演算法當場生成，難度與使用者實力都由計算得出，並依實力調整下一題。

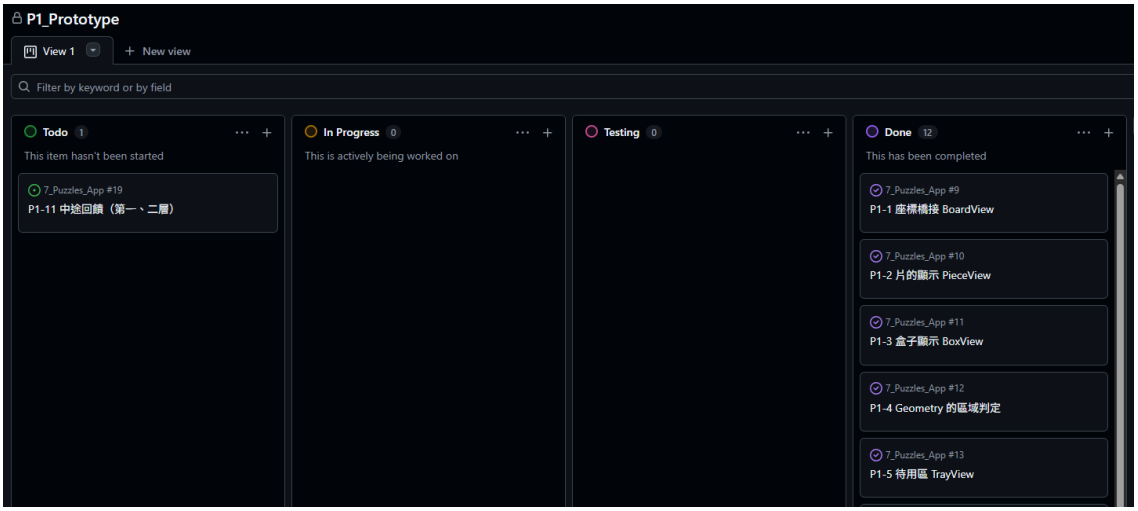
選擇用敏捷的方式開發，是因為**核心的三件事都沒有現成答案**——題目能不能保證拼得出來、形狀的難度怎麼量、實力怎麼換算，都要做出來、量過才知道可不可行。所以開發不是照一份完整規格往下做，而是**每次只承諾一小段、做完就驗證，再決定下一段**。

二、準備產品待辦清單

產品待辦清單放在 GitHub Projects，每個開發階段一個看板（P0 專案骨架、P1 可操作雛形、P2 核心演算法），欄位是 Todo → In Progress → Testing → Done。每一項工作是一張 issue 卡片，卡片上寫兩樣東西：

- **驗收句**：用「使用者看得到的結果」定義完成，而不是模組名稱。例如 P1 的完成定義是「七片都能從待用區拖進盒子」「靠近合法位置會明顯吸住，而且吸不出盒子外」。
- **收手點**：時間不夠時停在哪裡，作品仍然可以展示。例如求解器切成三塊，每一塊都能單獨停下。

待辦清單是會變的。P2 開工時重新切成 15 張卡；**9/10 發現初賽影片需要「過關 → 實力變化 → 下一題」的完整循環**，於是把原本排在 P3 的實力值（ELO）三張卡提前進 P2，並同步重排後面所有階段的工時。



圖一：GitHub Projects 的 P1（可操作雛形）看板（節錄上半部）：12 張已移到 Done，「中途回饋」那張刻意留在 Todo（見五之①）。

三、決定使用的敏捷方法

以**看板方法**管理工作流動，並以**開發階段當作衝刺**（P0 → P1 → P2，各約一週），每個階段結束時回顧、重估下一階段。每一張卡都走同一套流程：

表一：每張卡的流程

步驟	做什麼	為什麼
1. 討論	需要什麼功能、資料怎麼流、會漏掉哪些情況	先確定要解決的問題
2. 中文邏輯	寫程式之前，先用中文寫出「誰記住什麼、做什麼、發生什麼」	設計錯誤在這一步被抓到，比寫完程式再改便宜
3. 確認	中文邏輯確認過才動手	避免照著沒想清楚的設計往下寫
4. 實作與自動化測試	程式與測試一起交付	每張卡都要有會變紅的測試
5. Testing 欄	實機玩過才從 Testing 移到 Done	手感與版面問題只有實機看得到

三個支撐這套流程的做法：

- **完成的定義包含測試**：全部自動化測試通過才算完成。核心程式不依賴遊戲引擎，所以測試可以在引擎外執行，**幾十秒就有回饋**，每一次修改都跑得起全套。
- **程式與卡片連在一起**：commit 訊息附上卡片編號，issue 上會留下每一次提交的連結；程式寫完先進 Testing，**實機驗過才由人移到 Done**（圖二）。
- **待辦與決策各只有一個家**：還沒做的事、待決的問題、技術債集中在同一份待辦清單；已定案的決策另外記下理由。每天從那份清單開始工作。

四、追蹤專案進度

進度用看板欄位追蹤，另外做了兩件事讓「規劃」與「實際」可以對照：

預定日期訂了就不回頭改。 9/11 替 P2 每張卡訂了預定日期，規定「只准補實際，不准改預定」，這樣規劃與實際的差距才看得出來。

表二：P2 的規劃與實際（節錄）

卡	預定	實際
難度權重與尺度	9/12 ～ 9/14	9/12（提前 2 天）
照難度生成題目	9/15 ～ 9/16	9/12（提前 3 天）
局內統計、實力值（ELO）	9/17 ～ 9/19	9/12 ～ 9/13
監控面板	9/20 ～ 9/24	9/14

領先的時間怎麼用，也是一個決定。 進度領先約一週時，沒有拿去做延後也不影響初賽的求解器，而是**把找真人試玩、寫文件提前**——因為初賽只看文件與影片。

測試數量跟著功能成長：124（9/9，P1 收尾）→ 423（9/14）→ 470（9/15）→ **495（9/23）**。



圖二：issue #13（待用區抽屜）的時間軸：卡片移到 Testing →「尚未實測」的提交 → 實機測過移到 Done。

五、學習和調整

每一則都是同一個格式：**原本的決定 → 做下去或拿給人看 → 改成什麼**。

① 開工前先驗證前提，整張卡被重新規劃。

「卡住時的中途回饋」原本的規格寫著可以用一個很便宜的方法判斷。動手前先寫測試驗證它的前提——「合法的擺放一定對齊固定網格線」——結果 392 個合法擺放裡只有 41 個對齊（10.5%），**前提是假的**。這張卡因此從 P1 移到 P2，併進求解器那條線；原本的 issue 刻意留著不關，保留卡片改變形狀的紀錄。**代價是一組測試，不是一個做壞的功能。**

② 刻意提前的驗證，兩天抓到四個不會報錯的錯誤。

P2 重新切卡時，刻意插了兩張「提前驗證」：生十題實際玩、一次生一千題看分布，理由是不要等全部做完才第一次看到成品。結果兩天內找出四個不會出現錯誤訊息的問題，其中一個會讓**同一個形狀算出不同的難度**——如果沒抓到，會一路帶進難度權重與實力值，到那時只會看到「難度怪怪的」，很難回推到原因。



圖三：「生十題實際玩」那張卡的內文，開卡時就寫下「為什麼要有這張卡」。

③ 給人玩一次，推翻兩個已定案的設計。

回饋收集版給其他人試玩後，兩個有完整理由的定案被推翻：翻面鈕原本只給平行四邊形（三角形翻面等於多按幾次旋轉），但使用者覺得「少一顆按鈕很怪」，改成每一片都有、翻了沒變化時變灰；放棄一題後原本也問「難不難」，但有人放棄後選了「超簡單」，改成問「為什麼不玩了」。**兩次都保留了原本的理由，只是把它的範圍縮小。**

④ 先押注、再驗證：資料回來之後決定不改。

難度公式的權重在收集資料**之前**就定好，並寫成設定檔。之後 11 位使用者試玩 56 題，難度越高、提示用得越多，五個等級一路遞增；試過的調整最多只把相關係數從 0.456 提到 0.475，落在誤差內，所以**決定不改**。這個決定同時省掉了「改權重 → 重算換算倍率 → 重跑生成時間表 → 才能錄影」整條工作。

⑤ 把除錯的教訓變成規則。

專案另有一份除錯紀錄，專門記「不報錯、只是行為不對」的問題，至今 13 則，每則都要寫**怎麼發現的**與**失敗的嘗試**。它改變了之後的做法：一次畫面抖動連猜錯兩次原因之後，規則改成「先講清楚『如果是它，紀錄會長什麼樣』再去量」；另一次，待辦清單上早就記著的風險真的發生（一顆按鈕沒跟著隱藏，擋住了底下的縮圖），修法不是補那一顆，而是改成**整排按鈕一起隱藏**，讓之後新增的按鈕也不會再漏。

結語

這個專案的設計文件不是事先寫好的答案，而是**會被推翻的假說**：每一張卡都先寫下預期，再用測試、實機與真人試玩去驗證，被推翻就調整。看板與 commit 證明流程有在跑，上面五則證明**流程真的改變了決定**。